

When Extensions Lie: Large-Scale Study of Privacy Policy Compliance in Browser Extensions

Zilun Wang

The Chinese University of Hong Kong
Hong Kong SAR, China
zlwang22@cse.cuhk.edu.hk

Zeyang Zhuang

The Chinese University of Hong Kong
Hong Kong SAR, China
zyzhuang22@cse.cuhk.edu.hk

Mingxue Zhang

Zhejiang University
Hangzhou, China
Hangzhou High-Tech Zone (Binjiang)
Institute of Blockchain and Data
Security
Hangzhou, China
mxzhang97@zju.edu.cn

Wei Meng[✉]

The Chinese University of Hong Kong
Hong Kong SAR, China
wei@cse.cuhk.edu.hk

Michael R. Lyu

Chinese University of Hong Kong
Hong Kong SAR, China
lyu@cse.cuhk.edu.hk

Abstract

Browser extensions can pose significant privacy risks to end users due to their complex permission mechanisms and high privilege levels. Modern websites increasingly embed sensitive personal data within HTML elements, making such threats even more insidious. Although prior studies have examined browser extension privacy risks from different perspectives, they are typically constrained to specific sensitive data sources or a set of extensions tested on a few popular websites.

To enable a comprehensive compliance check of browser extensions' privacy practices at scale, we develop an automated dynamic analysis framework, EXT-POLICE. It applies LLM-based techniques to identify sensitive web content embedded in web pages, performs a multi-type taint analysis in the browser to comprehensively track sensitive data flows across browser components, and checks the compliance of such flows against the extension's privacy policy disclosures. We apply EXT-POLICE to evaluate 149,216 Chrome extensions across 7,200 unique web pages and identify 2,246 extensions with policy violations, enabling scalable analysis of policy compliance for browser extensions. Our study uncovers thousands of policy violations potentially affecting up to tens of millions of users, highlighting major gaps in current extension-store review processes.

CCS Concepts

• **Security and privacy** → **Web application security**; **Browser security**.

Keywords

Browser Extension, Privacy Policy Compliance, Taint Analysis

ACM Reference Format:

Zilun Wang, Zeyang Zhuang, Mingxue Zhang, Wei Meng, and Michael R. Lyu. 2026. When Extensions Lie: Large-Scale Study of Privacy Policy Compliance in Browser Extensions. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE '26)*, October 12–16, 2026, Munich, Germany. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3832783.3837459>

1 Introduction

Nowadays, browser extensions have become an integral part of the web ecosystem. They significantly enhance user experience by providing various functionalities such as ad-blocking, password management, and content customization. To enable these capabilities, browser extensions are designed with high privileges. By declaring the necessary permissions, extensions can access both privileged JavaScript APIs and content in web pages. Privileged APIs enable access to browsing history, cookies, and network requests, exposing users to extensive privacy risks. Likewise, access to web content poses serious concerns, as modern websites increasingly embed sensitive information directly in their pages, amplifying the potential for policy violations. These features make privacy issues in browser extensions more complex, as they involve multiple parties—extensions, browser, and numerous different websites.

To mitigate these risks, major extension stores have strict requirements on extensions' privacy-relevant practices [8, 18]. They require extension developers to declare their data collection practices through both template-based *dashboard disclosures* and free-form *privacy policies*. Such disclosures can provide privacy transparency and help users make informed decisions about which extensions to install. However, verifying whether runtime behavior is consistent with these disclosures remains challenging. This creates an opportunity for policy violations—either due to developer oversight or intentional obfuscation. Many browser extensions have been observed to collect and use privacy-sensitive user data via both privileged browser APIs and access to sensitive web content, without sufficient disclosure [1, 2, 41, 48]. For example, Facebook reported in 2021 that several malicious browser extensions scraped users' profile information directly from the web pages [48]. These



This work is licensed under a Creative Commons Attribution 4.0 International License. ASE '26, Munich, Germany

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2882-2/2026/10
<https://doi.org/10.1145/3832783.3837459>

extensions claimed they did not collect any personal information in their privacy policies, misleading users into installing them.

Although prior works [32, 47] have examined policy compliance of browser extensions, they do not provide a systematic analysis from an extension-specific perspective. Ling *et al.* [47] analyzed extensions against general privacy regulations rather than policy requirements specific to browser extensions. ExtPrivA [32] used a network traffic-based method to analyze the compliance of browser extensions with their privacy statements, but it could not observe data retained only in local storage and covered only four high-level data types using rule-based traffic analysis. More importantly, neither work considered various sensitive data embedded in web pages, which constitutes a major source of privacy risks. As discussed above, browser extensions can access sensitive data from web pages, which is increasingly prevalent in modern websites. Arcanum [62] first accounted for privacy risks arising from web content. It requires manually labeling sensitive data as sources for taint analysis, which limits the analysis to a small set of websites. Moreover, many browser extensions (47.5% as shown in §4.1) expose functionalities only on specific hosts declared in their manifest files, collectively covering more than 66K unique domains. To measure the privacy risks in such extensions, we need to monitor their behaviors across specified hosts at scale, which is infeasible for manual labeling. In addition, Arcanum and other prior works [38, 39, 57] primarily focus on data collection behaviors, without distinguishing types of sensitive data. Consequently, they cannot be readily applied for policy compliance analysis.

In this work, we aim to analyze the privacy policy compliance for browser extensions at large scale to uncover potential policy violations and thus protect users' privacy. We face the following two key challenges. Firstly, modern websites commonly embed diverse sensitive data in different contexts. It is challenging to precisely capture the semantics of such data and determine the sensitivity (C1). Secondly, to analyze the consistency between data collection behaviors and the privacy statements, we need to not only track the sensitive data flows across various complex components within the browser, but also associate each flow with its sensitive data types, which is technically challenging (C2).

To address these challenges, we present EXTPOLICE, an automated and scalable framework for policy compliance analysis of browser extensions. Specifically, to address C1, EXTPOLICE leverages large language models (LLMs) to analyze the data semantics and automatically identify sensitive content from web pages, enabling a scalable and consistent analysis. To address C2, it features a multi-type dynamic taint analysis engine, tailored for policy compliance checking and capable of tracking tainted data flows for multiple sensitive data types across the browser components. Building on these capabilities, EXTPOLICE can automatically analyze the privacy policy compliance of browser extensions by cross-checking the observed data collection behaviors with the extracted privacy statements for corresponding data types. In particular, EXTPOLICE is capable of monitoring the browser extensions' sensitive data access behaviors on the declared hosts in the manifest file, thereby significantly expanding the investigation scope of potential privacy policy violations to cover all extensions. In summary, compared with prior work, EXTPOLICE integrates LLM-based sensitive content identification, multi-type runtime taint analysis, and privacy policy

analysis, enabling automated and scalable compliance checking of extensions across diverse web pages.

We used EXTPOLICE to analyze policy compliance for 149,216 (out of 164,426) Chrome extensions (62,900 out of them with exclusive hosts declared) across 7,200 unique web pages. We observed 2,515 extensions that collected sensitive user data and identified 2,246 (89.3%) with policy violations under the Chrome Web Store's disclosure requirements, potentially affecting up to 61.6 million users. Moreover, when analyzing the privacy statements of the 2,515 extensions, we found that 21.4% of them did not provide either dashboard disclosures or privacy policies, and 61.0% of them had contradictory statements in the dashboard disclosures and privacy policies. The above results demonstrate that EXTPOLICE can automatically and effectively identify privacy policy violations of browser extensions at scale. The study also highlights the urgent need to enhance the extension stores' review process of browser extensions. We responsibly disclosed all the flagged extensions to the Chrome Extensions team via private channels, who noted that the cases were under review but had no further updates at the time of writing.

In summary, this paper makes the following contributions:

- We present EXTPOLICE, an automated framework for analyzing the privacy policy compliance of browser extensions at scale.
- We propose a novel multi-type dynamic taint analysis engine that distinguishes sensitive data types, which can be applied to broader compliance analysis.
- To the best of our knowledge, we are the first to apply LLMs to automatically annotate sensitive web content, demonstrating its effectiveness at scale and utility for web privacy research.
- We evaluated EXTPOLICE on ~150K extensions and revealed thousands of previously undetectable privacy policy violations.

2 Problem Statement

2.1 Background

2.1.1 Browser Extension Permissions. Browser extensions consist of content scripts and service workers¹. Content scripts can access and modify the content in web pages, while service workers have access to privileged browser APIs (e.g., history, cookies, and network request APIs). These capabilities must be declared in a manifest file, and apply only to hosts specified by host match patterns. Such match patterns are defined in the `host_permissions` field and the `content_scripts.matches` field, to specify the list of hosts or domains on which an extension can use privileged APIs or inject content scripts. In the following, we refer to such extensions as *specific-host* extensions, as opposed to *generic-host* extensions that match all URLs (e.g., `<all_urls>` or `*/.*`) or omit host permissions entirely.

2.1.2 Privacy Requirement for Browser Extensions. Major extension stores [9, 12] require developers to provide clear disclosures of their privacy practices. Developers of extensions distributed through the Chrome Web Store should disclose their privacy practices through (1) a template-based *dashboard disclosure*, which describes data practices using a fixed set of nine predefined data categories (see

¹Or background scripts in Manifest V2, which has been largely migrated to Manifest V3 [4, 7]. At the time of writing, Manifest V2 has been deprecated in Chrome.

Table 1: Privacy-Sensitive Data Types, Examples, and Corresponding Taint Sources.

Data Type	Examples or Description [†]	Taint Sources in ExtPOLICE
<i>PII</i> *	Name, address, email, address, age, identification number	Sensitive web content
<i>Health Info.</i>	Heart rate data, medical history, symptoms, diagnoses, procedures	Sensitive web content
<i>Financial Info.</i>	Transactions, credit card numbers, credit ratings, financial statements, payment history	Sensitive web content
<i>Auth. Info.</i>	Passwords, credentials, security question, personal identification number (PIN)	Sensitive web content, <code>document.cookie</code> , <code>chrome.cookies</code> , cookies in <code>chrome.webRequest</code>
<i>Personal Comm.</i>	Emails, texts, chat messages	Sensitive web content
<i>Location</i>	Region, IP address, GPS coordinates, information about things near the user's device	Sensitive web content, Geolocation API, IP Address in <code>chrome.webRequest</code>
<i>Web History</i>	The list of web pages a user has visited, with associated data such as page title and time of visit	<code>chrome.history</code> , <code>chrome.tabs</code> , URL, Domain, Title, URL in <code>chrome.webRequest</code> / <code>chrome.webNavigation</code>
<i>User Activity</i>	Network monitoring, clicks, mouse position, scroll, keystroke logging	Keyboard events, <code><input>.value</code>
<i>Web Content</i>	Text, images, sounds, videos, hyperlinks	Hyperlinks in <code><a></code> , randomly selected elements

[†] The data types and examples/descriptions are adapted from the Chrome Web Store Developer Dashboard [18].

* PII denotes personally identifiable information.

Table 1); and (2) a free-form *privacy policy*, which is an externally hosted document referenced via a URL on the extension's overview page [18]. Chrome explicitly states that any discrepancy between the runtime behavior, *dashboard disclosure*, and *privacy policies* constitutes a violation of the Chrome Web Store developer program policies. While Firefox enforces similar disclosure requirements [8], we focus on Chrome due to its dominant market share and widespread adoption [6, 29].

2.2 Research Problem

In this work, we aim to analyze policy compliance of browser extensions in an automated and scalable way. Compliance can be assessed by analyzing and comparing the extension runtime behavior—especially the types of data collected at runtime—with the corresponding policy statements in two types of disclosures. However, existing policy compliance analysis techniques exhibit limited effectiveness. This is because analyzing the privacy-related runtime behavior of browser extensions is inherently challenging: while extensions access sensitive data through privileged APIs, they can also directly read the content of web pages, introducing a distinct and largely unexplored dimension of privacy risk. Web pages often contain a wide range of highly sensitive information (e.g., credentials, contact information, and payment details) [49], making them a critical source of privacy-sensitive data that should be considered for comprehensive compliance analysis of browser extensions. However, prior works [32, 47] have largely overlooked this source of privacy risks, resulting in incomplete assessment of privacy policy compliance.

A common approach to addressing this gap is to manually annotate sensitive content on a small set of web pages, and analyze the relevant runtime data flows between the extension and these pages [62]. However, this approach does not scale, as a substantial portion of extensions expose functionalities only on their declared hosts (i.e., *specific-host* extensions as defined in §2.1.1). According to our measurement, over 78K out of 164K extensions (~48%) in Chrome Web Store are *specific-host* extensions, collectively declaring 175K hosts across 66K unique domains (§4.1). Moreover, since extensions' behaviors are tightly coupled with site-specific DOM

structures, they cannot be faithfully evaluated on synthetic or arbitrary pages. As a result, effective compliance analysis of *specific-host* extensions requires testing them on their declared hosts and identifying sensitive content on those pages, rather than relying on a small set of popular websites. This introduces a key challenge: scalable identification of sensitive content and accurate categorization of data across diverse web pages.

To solve the above issues, we aim to design a policy compliance analysis framework that *automatically* and *scalably*: i) identifies sensitive data types specific to browser extensions, particularly sensitive content embedded in web pages, ii) captures runtime collection behaviors related to such data while distinguishing the data categories, and iii) extracts the privacy statements from the two policy sources to check the consistency with actual behaviors.

2.3 Prior Works and Limitations

2.3.1 Policy Compliance Analysis of Browser Extensions. As mentioned in §2.2, prior works do not systematically account for privacy risks from sensitive web content, which limits their ability to support extension-specific policy compliance analysis. ExtPrivA [32] performs network traffic analysis to capture data sharing practices of extensions on real-world websites, and identifies inconsistencies with extension-specific privacy disclosures. However, it cannot systematically identify sensitive data embedded in web pages, as its rule-based heuristics (bait values and hyperlinks) cover only a limited set of data types, failing to capture many types of sensitive data (e.g., Personally Identifiable Information, Health Information, Financial Information) that are commonly present in web pages and highly relevant to extension privacy risks [1, 48]. Ling *et al.* [46, 47] study extension privacy practices from a GDPR compliance perspective, focusing on general legal requirements rather than extension-specific disclosures that our paper studies. Moreover, that work treats browser extensions as generic applications without modeling their interactions with web pages. Some other works [38, 49, 57, 60, 62] study privacy leakage or runtime behaviors of browser extensions. However, they commonly focus on detecting the presence of specific data flows without incorporating data semantics or policy compliance analysis.

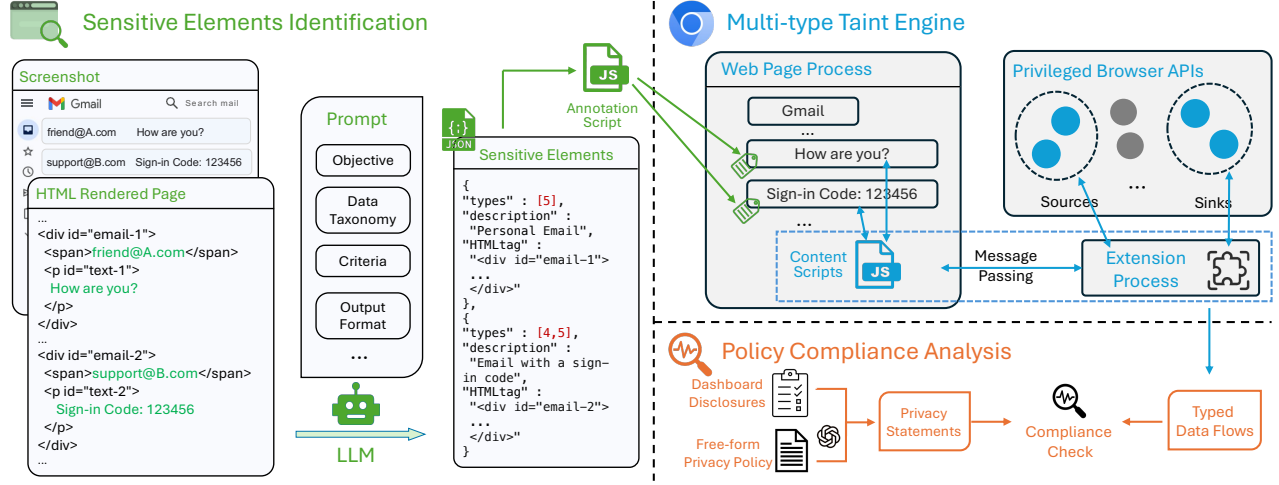


Figure 1: Overview of EXTPolICE.

2.3.2 Runtime Behavior Analysis of Browser Extensions. A key challenge for privacy policy compliance study is how to effectively and comprehensively analyze runtime behaviors of browser extensions, particularly the data flows between extensions and visited web pages. Prior works propose various techniques, such as static analysis [39, 47], network traffic inspection [32, 57, 65], and dynamic taint tracking [37, 38, 61, 62]. Static analysis is lightweight but imprecise, as it may overlook dynamically invoked APIs and flag API appearances without confirming that sensitive data is actually collected. Network traffic analysis can capture actual data exfiltration but cannot handle hashed or encrypted data, nor can it identify cases where extensions merely store privacy-sensitive information in local storage. Taint analysis is a promising technique for tracking data flows at runtime, but most prior works did not systematically model sensitive data flows from web content. The recent work Arcanum [62] first systematically analyzes data flows between extensions and web content on visited websites, but it relies heavily on manual annotation of sensitive elements on a small set of websites, limiting the scalability and coverage. In addition, existing taint analysis techniques do not distinguish the semantic types of sensitive data—especially that embedded in visited websites—making it difficult to determine whether observed data flows align with the types of collected data disclosed in privacy policies.

2.3.3 Other Related Work. Prior work has analyzed privacy policy compliance in mobile apps and websites [26, 27, 31, 33, 54, 66, 67]. Recent studies further apply LLMs to general policy compliance tasks [40, 63]. However, these approaches target general applications: they do not model browser extensions’ interactions with visited web pages or compare typed runtime flows with extension-specific disclosures. Beyond policy compliance, prior research has examined extension-relevant vulnerabilities [35, 36, 42], fingerprinting [24, 45, 58], vulnerable data flows [39, 56, 64], and malicious-extension detection [25, 44, 52]. These studies address broader extension-security risks, whereas our work evaluates whether observed data collection is consistent with extension-specific privacy disclosures.

3 Design

In this section, we present the design and implementation of EXTPolICE, an automated framework for analyzing policy compliance of browser extensions on large-scale web pages. The architecture of EXTPolICE is shown in Figure 1. EXTPolICE is designed to address two key challenges in automated and scalable browser extension compliance analysis. 1) *Automated* identification of sensitive web content. EXTPolICE leverages LLM-based analysis to automatically identify sensitive content embedded in web pages and classify it into predefined data categories, enabling large-scale evaluation of *specific-host* extensions across various web pages. 2) *Type-aware* tracking of runtime data practices. Unlike traditional taint engines that treat all sensitive data uniformly, EXTPolICE preserves the data types during taint tracking, allowing it to capture what categories of sensitive data are collected. This is essential for scalable compliance analysis, as it eliminates the need for inferring data types from tainted data flows, which is error-prone and requires manual effort. Building on these capabilities, EXTPolICE extracts privacy statements from both dashboard disclosures and privacy policies, and compares them with the observed type-aware tainted data flows to analyze policy compliance. By combining automated sensitive data identification, type-aware data flow tracking, and policy-aware analysis, EXTPolICE enables scalable compliance analysis across a large number of extensions and web pages. In the following, we introduce each component of our framework in detail.

3.1 Sensitive Web Content Identification

We develop an LLM-based approach to identifying and categorizing sensitive web content. As discussed in §2.2, this task is a fundamental yet non-trivial prerequisite for analyzing runtime data practices over sensitive web page content. It requires reasoning over both the semantic meaning of page content and its surrounding context, including labels, headings, and visual layout, as well as mapping the identified content to appropriate DOM elements. Such requirements are difficult to satisfy using manual annotation or rule-based and traditional NLP methods. To this end, EXTPolICE leverages

LLMs and adopts a hybrid input design that combines the *rendered HTML page* with a *screenshot* of the rendered page, capturing both structural and visual information, as illustrated in [Figure 1](#).

3.1.1 Annotation Criteria. Annotating sensitive web content is inherently subjective and lacks standardized ground truth, often leading to inconsistencies across human annotators. To ensure consistency and reproducibility, we define a set of operational criteria that guide both LLM-based annotation and manual validation.

User-Associated Data Annotations are restricted to data directly associated with the current user, reflecting actual privacy risks during browsing. We exclude non-personal content (e.g., disclaimers, footers, news, advertisements), and unrelated third-party data (e.g., other users' profiles). Input elements intended for sensitive data (e.g., login or email fields) are labeled based on their functional role, even if unpopulated.

Multiple Data Categories HTML elements containing content relevant to multiple categories are annotated with all applicable data types. This accounts for overlapping semantics in real-world content. For example, an email widget may include diverse types of private data depending on the message content—ranging from personal communications to financial information (e.g., invoices or transaction records), or authentication-related data (e.g., password reset links or verification codes). Such content should be labeled with all relevant categories to accurately reflect the privacy implications.

Fine-grained DOM annotation Sensitive content is associated with HTML elements, and taint tracking operates on labels assigned to these elements. Annotations are applied to the innermost elements that fully encapsulate the relevant data, excluding container elements that introduce unrelated content or serve only structural or styling purposes. When all sibling elements contain the same data type(s), annotation is lifted to their nearest common ancestor, provided it does not include non-sensitive content.

3.1.2 Prompt Design. To enable LLMs to reliably identify sensitive content on web pages, we design a structured prompt template that incorporates the annotation criteria above. To ensure consistency and task focus, the template leverages few-shot [30] prompting technique and persona assignment [53]. It specifies the data taxonomy, annotation criteria, detailed instructions, output format, and illustrative examples. The full prompt template is available in our artifact.

As shown in [Table 1](#), six of the nine categories are involved in the LLM-based sensitive content identification. We provide the LLM with the rendered HTML page and its screenshot as input, which allows the LLM to understand the web content and layout. To ensure the response from the LLM can be automatically processed, we define the output format explicitly. For each piece of identified sensitive data, the LLM provides three fields: 1) *types*, a list of predefined sensitive data categories specified in the prompt; 2) *description*, a textual explanation facilitating our manual review of the output; and 3) *HTMLtag*, an HTML snippet that precisely contains the identified sensitive data. Following our annotation criteria, the LLM identifies sensitive data with multiple applicable categories and maps them to the corresponding elements.

[Figure 1](#) shows a simplified example on the Gmail website, illustrating how sensitive elements are identified and mapped to specific HTML tags. Both emails in the example are identified as *Personal Communications*, while the second is additionally classified as *Authentication Information* because it contains a sign-in verification code. In this case, both `<span>` and `<p>` tags include the sensitive data, and their parent tag does not include other non-sensitive data, so the returned HTML tag should be their closest shared parent element (as shown by `<div id="email-1">...</div>`). This granularity ensures a precise annotation that directly targets sensitive data without introducing excessive or irrelevant content into the taint tracking.

3.1.3 Web Page Preparation. While nowadays LLMs support large context windows (e.g., 400k tokens for gpt-5.2), HTML documents from complex websites (e.g., amazon.com, yahoo.com) still exceed the context limits. To address this, we preprocess HTML code by removing non-essential tags and attributes, including functional or styling elements (e.g., script, style, meta) and inline event handler attributes (e.g., onerror, onclick). This reduces input size while preserving the semantic content of the page.

To ensure consistency between LLM annotations and runtime analysis, EXT-POLICE adopts a record-and-replay strategy. Using WprGo [11], we capture web pages with their DOM structure and screenshots for offline annotation. During replay, EXT-POLICE injects scripts that assign data-taint attributes to identified sensitive elements via XPath [20], enabling precise alignment with taint tracking. This design ensures consistent page states across analysis stages and avoids redundant network requests or unintended side effects.

3.2 Multi-Type Dynamic Taint Analysis

As discussed in §2, analyzing browser extensions' runtime behaviors requires tracking sensitive data flows across both web content and privileged browser APIs. Specifically, the analysis should differentiate sensitive data across multiple categories to enable precise compliance checking against policy disclosures, which is beyond the capabilities of existing taint engines. To address this gap, we develop a multi-type taint engine tailored for browser extensions' policy compliance analysis, which tracks distinct types of sensitive data across the rendering engine (e.g., Blink [10]) and JavaScript engine (e.g., V8 [19]). Our design is based on Chromium browser and built upon Arcanum [62] with ~2k LoC modifications to support multi-type taint tracking. We note that our approach can be adapted to other browsers with similar extension architectures, such as Firefox and Edge.

3.2.1 Design Principle. A multi-type taint engine should explicitly distinguish and track multiple categories of privacy-sensitive data, so that each data flow can be associated with its specific type(s) at sinks for compliance checking. We use the term *typed taint* to denote taint metadata that encodes these data categories, rather than the conventional binary tainted/untainted label. Typed taints are assigned at sensitive data sources, uniformly represented across web content and privileged APIs, and propagated across browser components. They should be preserved and updated during complex data operations, such as string manipulations and composite object

handling. Although our taint engine builds on Arcanum and inherits its basic taint update and propagation mechanisms, EXTPOlice does not merely enlarge the taint field to accommodate multiple data types. Instead, to meet the requirement discussed above, we redesign the taint representation and propagation logic to preserve, merge, and separate different categories of sensitive data across strings and composite objects. We also introduce new taint sources for *User Activity*, an important data type defined in the dashboard disclosures. We detail these new designs below.

3.2.2 Representation of Typed Taint. To uniformly represent multiple sensitive data types, EXTPOlice uses a compact bitmask, `typed_taint`, where each bit corresponds to one category. This enables a tainted value to carry multiple categories simultaneously, and supports efficient updates through bitwise operations. To minimize the overhead, we implement `typed_taint` as an unsigned 8-bit integer. All nine data categories fit within this space, with one bit shared by *Health Information* and *Financial Information*, as these types are rarely observed in practice (see §4.2).

In modern browsers, sensitive data may flow through both the rendering engine (e.g., Blink) that manages web content operations, and the JavaScript engine (e.g., V8), which accesses the privileged browser APIs. To support end-to-end in-browser taint tracking, EXTPOlice associates `typed_taint` metadata with runtime objects in both engines. Following prior browser taint-tracking systems [38, 62], EXTPOlice adopts different integration strategies for Blink and V8 because they represent and manage runtime objects differently. In Blink, where the native objects have stable layouts, we extend the objects with an additional `typed_taint` field, enabling fine-grained tracking during DOM operations. In V8, EXTPOlice maintains `TypedTaint Table`, a global hash table keyed on the addresses of tainted runtime JavaScript objects, to store `typed_taint`. The taint representation provides per-byte granularity for strings and per-element granularity for composite objects such as arrays and maps. Figure 2 illustrates an example of typed taint representation for a map object `rst` in an extension’s simplified code. The value corresponding to key `mails` is a concatenated string containing two emails on Gmail (as shown in Figure 1, each associated with a different taint type). With the fine-grained design, only the parts from the `elem.text` are tainted with the corresponding types, while the inserted semicolons are not tainted. Similarly, the key `history` maps to data returned by the `chrome.history.search` API, which is assigned the *Web History* type. With this design, EXTPOlice can distinguish various types of sensitive data within `rst`, rather than treating the entire object as uniformly tainted.

3.2.3 Multi-type Taint Propagation. Unlike binary taint tracking—where the propagation is trivial by tainting any output derived from tainted input—type-aware tracking requires finer precision to capture how different types of data are combined or nested within complex data structures. For example, composite objects like arrays or maps require fine-grained taint that preserves type information for each element.

For string objects, instead of associating a single `typed_taint` with the entire object, EXTPOlice maintains a vector of `typed_taint`, one per byte, to achieve byte-level granularity. This prevents imprecise propagation: for example, when a string tainted as *PII* is concatenated with another tainted as *Web History*, the

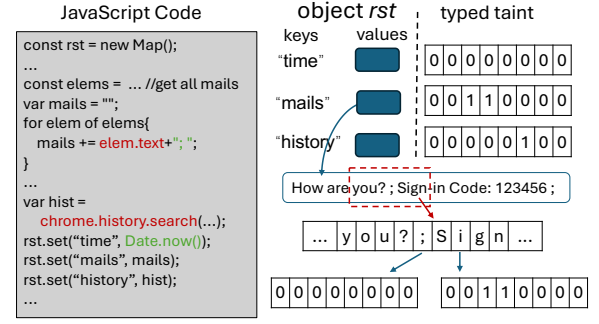


Figure 2: An example of typed taint representation.

resulting string contains both taint types. Instead of setting the entire string’s typed taint as $PII \wedge Web\ History$, EXTPOlice retains the typed taint for each byte. Otherwise, subsequent operations on substrings could cause over-tainting and false positives. All relevant operations on these objects are instrumented to ensure that the typed taints are updated consistently. To reduce runtime overhead, EXTPOlice adopts a lazy-update strategy. Uniformly tainted values are initially stored as a compact length-1 vector. When later operations demand finer granularity, this compact form is expanded into a full per-byte vector.

For composite objects such as Maps or Arrays, EXTPOlice maintains typed taints at the element level, so that each entry preserves its own `typed_taint`. The taint type of the container object is defined as the union of its elements’ types and is dynamically updated as elements are added, removed, or modified. This ensures the correct propagation of typed taint for composite objects: when a specific element from a Map or Array is retrieved and assigned to a new variable, only that element’s `typed_taint` is propagated, rather than the union of all elements. The same principle applies recursively to nested objects, where each element maintains its own taint metadata, and the container’s typed taint reflects the union of all descendants.

3.2.4 Taint Sources. As shown in Table 1, EXTPOlice assigns typed taints to two primary classes of sources: 1) sensitive web content embedded in DOM elements, and 2) data accessed through privileged browser and extension APIs.

Sensitive Web Content Sensitive data on web pages is first identified offline using the LLM-based approach (§3.1). At run time, EXTPOlice annotates these elements with a `data-taint` attribute, enabling Blink to assign the corresponding typed taint upon access. To capture indirect data access (e.g., via `innerHTML`), we instrument relevant DOM APIs to propagate taints from child elements. We further ensure that taints are consistently applied within the DOM hierarchy by assigning the same `typed_taint` to all descendants of a tainted element.

Because the *Web Content* category overlaps with more specific data types derived from DOM elements, EXTPOlice assigns those specific categories whenever applicable and reserves *Web Content* for generic page content. Exhaustively tracking all DOM elements as *Web Content* is prohibitively expensive for large-scale analysis. Therefore, EXTPOlice assigns the *Web Content* taint to all hyperlinks (as in ExtPrivA [32]), and up to k randomly selected elements

from each recorded page, with $k = 3$ by default. We exclude non-content elements (e.g., script, style, and noscript) and elements already assigned a more specific sensitive-data type. When an ancestor and a descendant have identical normalized rendered text, we retain only the deeper element to avoid redundant candidates. This strategy enables EXT_{POLICE} to detect potential page-content leakage with bounded analysis overhead, as illustrated in §4.5.

Sensitive Data via Privileged APIs For data accessed via privileged browser and extension APIs, we instrument sensitive APIs to assign typed taints to their return values based on the API semantics. For each data category, we consider both direct and indirect access paths. For example, *Web History* can be obtained not only through the `chrome.history` API, but also indirectly via the `chrome.tabs` API or injected content scripts that extract page metadata such as URLs and titles. Similarly, EXT_{POLICE} incorporates the *User Activity* type by tracking user input flows, which is not considered by prior taint engines. We handle both direct access via `input.value` and indirect access through `KeyboardEvent` APIs [15] (e.g., `keyup`, `keydown`, and `keypress`), which are commonly used to capture user input. During runtime analysis, we simulate the user input by sending keys to the input fields in the tested web pages. Specifically, we also consider cookies as a source of sensitive data. Although cookies may include diverse sensitive data (e.g., *PII* and *Location*), we conservatively classify them as Authentication Information, since session tokens and login credentials are typically among their most security-critical contents.

3.2.5 Taint Sinks. Following the design of Arcanum, EXT_{POLICE} supports taint sinks including local storage, web requests, and DOM element injection. Web requests and local storage align directly with Chrome’s definition of data collection practices [18]. For DOM element injection, we specifically track cases where tainted data is written into `src` attributes, which may trigger external resource loading or third-party requests and thus leak sensitive information. When a tainted data flow reaches a sink, EXT_{POLICE} records the object’s merged `typed_taint` values for subsequent policy compliance checking.

3.3 Policy Compliance Analysis

As noted in §2.1.2, the Chrome Web Store explicitly treats discrepancies among runtime behaviors, *dashboard disclosures*, and *privacy policies* as violations of its developer program policies. ExtPrivA [32] compares runtime data flows with dashboard disclosures and identifies contradictions between privacy statements, but does not compare runtime behaviors with statements in privacy policy documents. We use dashboard disclosures as the primary basis for determining whether observed data collection is disclosed, and use privacy policies as complementary evidence. Our analysis identifies two types of compliance issues: *runtime behavior violations*, where observed data collection is insufficiently disclosed, and *privacy statement contradictions*, where different sources make incompatible claims about the same type of data.

3.3.1 Compliance Model. EXT_{POLICE} normalizes runtime data flows, dashboard disclosures, and privacy-policy statements to Chrome’s nine sensitive-data types, denoted by $\mathcal{T}$. Let $\mathcal{R} \subseteq \mathcal{T}$ contain the types observed in tainted data flows that reach a sink.

We represent dashboard and privacy-policy statements as sets of tuples S_D and S_P . Each tuple $s = (t, a)$ pairs a sensitive-data type $t \in \mathcal{T}$ with an action $a \in \{True, False\}$. The action affirms (*True*) or denies (*False*) the collection of data of that type. Let $\mathcal{D} \subseteq \mathcal{T}$ be the set of types listed in the dashboard disclosure. Then, $S_D = \{(t, True) \mid t \in \mathcal{D}\} \cup \{(t, False) \mid t \in \mathcal{T} \setminus \mathcal{D}\}$. For a privacy policy, EXT_{POLICE} uses an LLM to extract S_P and assigns $(t, False)$ when no relevant statement is found for type t [18]. Unlike dashboard disclosures, a policy may yield tuples with the same type but different action values.

Runtime Behavior Violations We derive the types for which collection is positively declared by the dashboard disclosure and privacy policy as $C_D = \{t \mid (t, True) \in S_D\}$ and $C_P = \{t \mid (t, True) \in S_P\}$. For each $t \in \mathcal{R} \setminus C_D$, we report a *runtime behavior violation*: data of type t is collected at runtime but not declared in the dashboard disclosure. We further classify the violation based on the privacy policy. The violation is *weak* if $t \in C_P$ (disclosed only in the privacy policy) and *strong* otherwise (disclosed by neither source).

Privacy Statement Contradictions A privacy statement contradiction occurs when different statements make incompatible claims about the collection of the same type of data. An *intra-policy contradiction* occurs when S_P contains tuples with the same type but different action values, i.e., $(t, True) \in S_P$ and $(t, False) \in S_P$. A *cross-source contradiction* occurs when S_D and S_P contain tuples with the same type but different action values, i.e., $(t, a) \in S_D$ and $(t, a') \in S_P$ for $a \neq a'$.

This category-level model enables direct category-level comparison across the three sources and captures only whether collection is affirmed or denied. It does not model purposes, conditional collection, recipients, or third-party sharing, which requires additional runtime context, including specific user inputs and execution conditions. We leave them as future work.

3.3.2 LLM-based Privacy Policy Analysis. We design a structured prompt to extract extension-specific privacy statements from policy documents. The prompt specifies a fixed data taxonomy of nine sensitive data types and instructs the LLM to identify text spans describing the extension’s privacy practices. For each extracted text span, the LLM produces one or more output tuples whose *Type* and *Action* fields correspond to t and a , respectively, and preserves the span as supporting evidence.

In practice, developers often mix extension-specific and non-extension privacy statements within the same policy, and may even provide only a generic website policy that does not describe the extension at all. As a result, accurately analyzing privacy policies requires identifying extension-related statements while filtering out irrelevant ones. Otherwise, the extracted statements may not faithfully reflect the extension’s privacy practices, leading to false positives. ExtPrivA [32] adopts a heuristic that retains only sentences containing the keyword “extension” for further analysis. However, this approach is brittle and leads to false negatives, as relevant statements may omit the keyword or refer to the extension indirectly (e.g., using its name or describing its functionality). To address these limitations, we leverage contextual understanding and require the LLM to determine whether a statement is related to the browser extension. Specifically, we consider both structural and

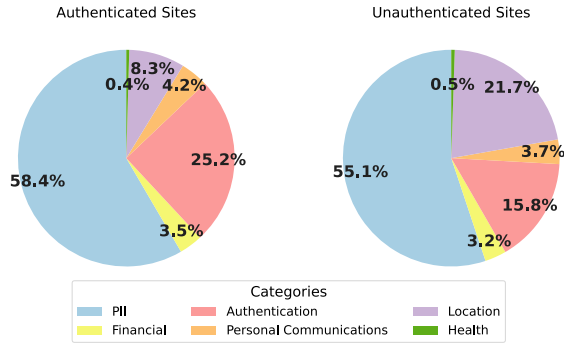


Figure 3: Category distribution of identified sensitive elements for two sets of target web pages.

semantic cues in the policy document. For example, if the policy includes a section that explicitly discusses the browser extension, we treat statements within that section as relevant to the extension. We also account for implicit signals, such as descriptions of extension-specific behaviors (e.g., collecting visited URLs or recording active tabs). By incorporating such contextual information, our approach avoids the limitations of keyword-based methods and enables more accurate extraction of extension-specific privacy statements.

4 Evaluation

We evaluate EXT POLICE on a large dataset of browser extensions and websites to study their compliance with their privacy statements. For each extension, we prepare target websites, identify sensitive content, track runtime data flows, and check policy compliance. We further assess the accuracy of our LLM-based modules for sensitive content identification and privacy policy analysis.

4.1 Experiment Setup

4.1.1 Extension Dataset. We constructed our dataset by crawling the Chrome Web Store’s sitemaps in March 2026, retrieving each extension’s metadata (e.g., user counts, external policy links, and dashboard disclosures) and source code. In total, we collected 164,426 valid extensions. Based on their manifest files, 47.5% (78,110) were *specific-host* extensions and 52.5% (86,316) were *generic-host* extensions. The *specific-host* extensions together declared 175K hosts across 66K unique domains.

4.1.2 Target Websites. Testing browser extensions requires different strategies depending on their host permissions. Accordingly, we design different target website settings for these two types. For generic-host extensions, which are not restricted to specific domains, we follow prior work [62] and evaluate them on a small set of representative popular websites. These sites include both login pages and post-login pages, enabling us to capture user interactions and privacy-sensitive content. Specific-host extensions must be tested on websites that match their declared domains. Ideally, this would require account creation and maintenance across a large number of websites, which is impractical due to the effort required. To cover diverse sensitive data types while maintaining scalability, we construct two complementary sets of target websites: *authenticated sites* and *unauthenticated sites*.

Authenticated Sites We select a set of representative popular websites to create test accounts that are commonly targeted by specific-host extensions and contain rich sensitive data. This set includes the same websites used for evaluating generic-host extensions. For each selected site, we include both post-login pages and their corresponding login pages, where login forms provide representative sources of *User Activity*. In total, we create test accounts on 59 websites and record 56 post-login pages and 48 unique login pages, yielding **104** pages.

Unauthenticated Sites Although EXT POLICE can automatically analyze arbitrary pages, we cannot analyze all web pages across all 66K domains declared in specific-host extensions due to resource constraints of LLM querying costs. Therefore, we use a best-effort way to construct a representative set of target websites to cover more extensions while keeping the evaluation feasible. We first filter the candidate domains within the Tranco Top-10K and remove inaccessible sites and duplicates that share the same web page. This step yielded 1,953 pages, which already cover over 40K extensions (51.7%). For extensions not covered by these domains, we additionally include the highest-ranked domain among their declared hosts, restricting candidates to domains within the Tranco Top-1M. Finally, we obtain **7,096** index pages, which cover 62,900 specific-host extensions (80.5% out of all specific-host extensions).

4.1.3 Large-Scale Testing. We implemented a prototype of EXT POLICE built on Chromium browser version 108.0.5359.71 and used GPT-5.2 for LLM tasks. In our experiments, we set the model temperature to 0.0 and used the default values of all other configuration parameters. Our multi-type taint engine is developed based on Arcanum [62]. We evaluated 149,216 extensions (86,316 generic-host extensions and 62,900 specific-host extensions) on two Debian servers (96 CPU cores and 500 GB RAM each), parallelized with 60 processes per server. We ran each extension for 45 seconds after the web page has been fully loaded locally through WprGo. The taint analysis phase was completed in 15 days.

4.2 Sensitive Content Identification

4.2.1 Result Overview. We apply our LLM-based methods to analyze sensitive DOM elements in both authenticated and unauthenticated sites. In total, we identify 425 sensitive elements across 104 authenticated sites (4.09 per site on average), and 14,952 elements across 7,096 unauthenticated sites (2.11 per site on average). Only 6 authenticated sites contain no sensitive elements, whereas 3,060 unauthenticated sites do not expose any sensitive content. On average, each page requires 10.02 seconds and costs \$0.059 for LLM queries. Figure 3 further breaks down the identified data types. PII was the most common, appearing on 947 web pages. By contrast, Financial and Health information was rarely observed, since such data is uncommon even after login and often requires real identities. For instance, we registered a real credit card for [paypal.com](https://www.paypal.com) to evaluate financial data, but did not include health information for ethical reasons.

4.2.2 Evaluation of LLM-based Annotation. Because no ground-truth dataset exists for sensitive-content identification, we constructed an evaluation set of 360 web pages (5% of all 7,200 pages). Given the much smaller number of authenticated pages, we included

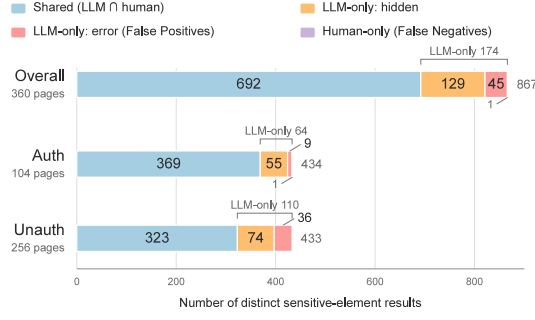


Figure 4: Comparison of LLM-based and human annotations of sensitive elements.

all 104 authenticated pages and randomly selected 256 unauthenticated pages. Two security researchers independently annotated sensitive elements following §3.1.1, with disagreements resolved through discussion. EXTPolICE identified 866 sensitive elements across the 360 sampled pages, while the annotators labeled 693. As shown in Figure 4, the LLM-based method identified 174 elements missed by the human annotators, whereas the annotators identified one element missed by EXTPolICE. We confirmed that the single element missed by EXTPolICE—an email input field—was a false negative. However, EXTPolICE correctly annotated this element in repeated runs, suggesting that the false negative arose from output variability. Among the 174 elements identified only by EXTPolICE, 45 were verified as false positives, whereas the remaining 129 were valid sensitive elements overlooked by the human annotators. The false positives mainly arise from overly broad interpretations, such as including non-sensitive website information or misclassifying benign input fields as sensitive. The valid additional elements often involve subtle or non-obvious patterns. For example, EXTPolICE can identify elements containing embedded user identifiers or CSRF tokens, which are difficult to detect through manual inspection. Overall, these results indicate that EXTPolICE complements manual annotation by improving coverage while introducing a limited number of false positives.

To evaluate the stability of the LLM-based annotations, we randomly sampled 10 unauthenticated and 10 authenticated websites and repeated the annotation process five times for each. Across these runs, the mean precision was $96.70\% \pm 1.42\%$, and no false negatives were observed, indicating consistently high recall with only modest variation in precision. We further assessed the contributions of the webpage screenshot and the annotation criteria by removing each component from the prompt. Removing the screenshot resulted in a comparable precision of 95.86%, but decreased the recall to 93.64%. This indicates that visual context helps EXTPolICE recognize sensitive elements that may be difficult to identify from HTML code alone. Removing the annotation criteria substantially decreased the precision to 39.01% and recall rate to 97.06%, demonstrating that the criteria are essential for constraining the model and preventing overly broad annotations.

4.3 Runtime Sensitive Data Flows

4.3.1 Result Overview. As shown in Table 2, we detected 1,279 generic-host extensions (1.48%) and 1,236 specific-host extensions

Table 2: Runtime Data Flows

Data Type	Generic-Host	Specific-Host
PII	33	12
Health Information	0	0
Financial Information	3	0
Authentication Information	34	46
Personal Communications	4	0
Location	28	2
Web History	1164	1131
User Activity	31	36
Web Content	744	998
Total Unique	1279	1236

(1.97%) with tainted data flows of sensitive data types. The most common types observed were *Web History* and *Web Content*. The large number of *Web History* flows may be because we considered both direct and indirect ways to collect web history (see §3.2.4). For example, an extension can access the URL of all visited pages to bypass the History API’s permission requirement. For *Web Content*, we take the sampled DOM elements and all hyperlinks on the website as taint sources, which makes the taint sources inherently large and thus more likely to be collected by extensions. We note that these results are still valid, as they reflect the extensions collecting such data. Even excluding these two types, there are still 95 generic-host and 91 specific-host extensions with tainted data flows of other sensitive data types.

4.3.2 Comparison with Prior Work. The dataset and source code of ExtPrivA are not publicly available, preventing a direct comparison with their runtime analysis. Nevertheless, since our multi-type taint engine builds upon Arcanum, we compare our results with their evaluation reported in March 2024. Arcanum evaluates extensions on a fixed set of seven popular websites, without considering the declared hosts of specific-host extensions. By excluding data flows observed on these websites, we identify 996 specific-host extensions with tainted data flows that would not be covered by their evaluation. Overall, we observe tainted data flows across 380 unique websites, demonstrating the importance of testing on extension-declared hosts at scale. In addition, EXTPolICE detects 67 extensions leaking *User Activity*, a data type not supported in Arcanum.

We further observe a slightly lower ratio of extensions with tainted data flows compared to Arcanum (2.68%). This difference is mainly due to their inclusion of additional data types (e.g., platform, device, and user agent), which are outside our nine-category taxonomy. Moreover, stricter enforcement by the Chrome Web Store (e.g., the deprecation of Manifest V2) may have reduced the prevalence of privacy-risky extensions in our dataset.

4.3.3 Manual Verification. We randomly sampled 50 extensions from each of the generic-host and specific-host groups, resulting in 100 extensions in total and 589 observed tainted data flows. For each extension, we manually examined the tainted data flows and assessed whether the exfiltrated data was genuinely privacy-sensitive. We identified 7 false positive data flows across 3 extensions, corresponding to a precision of 98.81%. These false positives stemmed from cases where the sensitive data was used for diverging the control flow, instead of being directly leaked via the sinks. For conservativeness, our taint engine also tracked such *implicit* data

Table 3: Privacy statement contradictions across different disclosure availability categories.

Disclosure Availability	#Extensions	Cross-source	Intra-policy
Only dashboard disclosures	215	215	N/A
Only privacy policies	662	250	167
Both present	1100	1070	302
Neither present	538	N/A	N/A
Total	2515	1535	469

flows. Note that in these cases, the sensitive data still indirectly affects the data leakage behaviors, and prior works Arcanum [62] adopted the same design. Importantly, the involved data categories also appear in other true positive cases, and thus these FPs do not impact the subsequent compliance analysis. Overall, the results confirm that our system reliably flags the collection and leakage of privacy-sensitive data on web pages.

4.3.4 False Negative Analysis. Because no comprehensive ground-truth dataset of extensions with privacy issues is publicly available, and most prior studies do not release complete extension lists, a systematic false-negative evaluation for our taint engine is infeasible. We therefore conducted a best-effort evaluation using extensions previously flagged by Arcanum [62] and 16 public security reports [2, 3, 5, 13, 14, 16, 17, 21–23, 28, 34, 41, 50, 51, 59]. These sources collectively covered 125 unique extensions which were reported with collection of sensitive user data. As extensions reported in the same post often implemented nearly identical malicious logic, we retained one representative extension per post, resulting in 26 extensions: 10 from Arcanum and 16 from public reports. Using the configuration described in §4.1, EXT POLICE successfully detected tainted data flows consistent with the reported behaviors in 14 extensions (9 from Arcanum, 5 from public reports). By manual inspection, we confirmed the 12 failed cases were caused by the following reasons. ① Unavailable remote dependencies (5 extensions). Malicious extensions often rely on *command-and-control* (C2) servers to dynamically deliver configurations or payloads that activate their malicious functionality. None of these 5 extensions received a valid response from its remote server, preventing the reported behavior from being triggered. As these extensions have been reported with malicious behaviors and removed from Chrome Web Store, one plausible explanation is that the corresponding remote infrastructure was no longer operational at the time of our experiment. We cannot determine whether these cases are false negatives of EXT POLICE. ② Unmet runtime conditions (6 extensions). These extensions required additional runtime conditions to activate their malicious behavior, such as specific user actions on target pages, interactions with the extension popup, or the expiration of timers. These conditions were not exercised by EXT POLICE in the experiment. ③ Unsupported data source (1 extension). This extension was reported to access clipboard content, which is not currently modeled as a taint source by EXT POLICE. The seven failures in ② and ③ therefore represent false negatives caused by complex runtime conditions and corner data-source case, which are not supported by EXT POLICE or prior works. We leave these improvements to future work.

Table 4: Policy Violations across Extension Types. v., s.v., and w.v. denote the numbers of extensions with violations, strong violations, and weak violations, respectively. The Users column reports the maximum number of potentially affected users, computed as the sum of users of all violating extensions.

Data Type	Gen. v. (s.v./w.v.)	Spec. v. (s.v./w.v.)	Users
PII	25 (22/3)	11 (10/1)	66.4K
Health Info.	0 (–/–)	0 (–/–)	–
Financial Info.	2 (2/0)	0 (–/–)	29
Auth. Info.	29 (24/5)	39 (29/10)	9.3M
Personal Comm.	4 (4/0)	0 (–/–)	452
Location	27 (24/3)	2 (2/0)	113.4K
Web History	985 (827/158)	1045 (922/123)	54.9M
User Activity	26 (24/2)	32 (30/2)	4.0M
Web Content	467 (394/73)	667 (590/77)	22.9M
Sum	1565 (1321/244)	1796 (1583/213)	–
Unique Ext.	1095 (889/206)	1151 (956/195)	61.6M

4.4 Privacy Policy Compliance Check

With the taint analysis results, we further check the privacy policy compliance of the flagged extensions, following the strategy described in §3.3. For each flagged extension, we extract privacy statements from its privacy policy using our LLM-based method to obtain $\mathcal{S}_P$, and collect dashboard disclosures to obtain $\mathcal{S}_D$.

4.4.1 Privacy Statement Contradictions. In this section, we present the combined results of the generic-host extensions and specific-host extensions together, as they show similar trends. In total, there are 1,315 out of 2,515 extensions with dashboard disclosures and 1,992 extensions with links to free-form privacy policies on their overview page. After further filtering out inaccessible links, we successfully crawled the free-form privacy policies for 1762 extensions. As shown in Table 3, we identify a total of 1,535 cross-source contradictions (*i.e.*, conflicting statements between $\mathcal{S}_D$ and $\mathcal{S}_P$) and 469 intra-policy contradictions (*i.e.*, conflicting statements within $\mathcal{S}_P$). We specifically inspected the 1,100 extensions providing both disclosure types, and found that 616 out of them had no privacy statement extracted from the free-form policies. By manually inspecting these cases, we find that most of these extensions provide invalid privacy policies, such as linking to error pages, pages unrelated to policies, or simply reusing policies of other platforms. This indicates that the Chrome Web Store lacks effective checks for the validity of the free-form privacy policies.

4.4.2 Policy Violations. Table 4 summarizes the detected policy violations across different data types and extension categories. Overall, we identify a substantial number of violations, with **1,095** generic-host and **1,151** specific-host extensions exhibiting at least one violation, potentially impacting up to **61.6 million** users. We further distinguish between *strong violations* and *weak violations* based on the consistency between runtime behaviors and *privacy policies*, as defined in §3.3. The distinction between the two types allows us to differentiate between completely undisclosed behaviors and those that are partially disclosed but lack transparency in structured disclosures. As shown in the table, only a small portion of the violations are weak violations, indicating that most of the observed privacy-violating behaviors were not disclosed in either source.

4.4.3 Evaluation of LLM-based Privacy Policy Analysis. Since no ground truth exists for the privacy statements in browser extension

policies, we sample 100 extensions with external policy links and manually inspect the privacy policies to verify the effectiveness of EXT*POLICE*. Overall, EXT*POLICE* extracted 228 privacy statements from these extensions. We then manually measure the false positives (mistakenly reported privacy statements) and false negatives (missed privacy statements), and find 9 false positives across 4 extensions and 6 false negatives on 2 extensions (precision: 96.05%, recall: 97.33%). The false positives primarily result from misinterpretation of certain statements. For example, descriptions of the collection of user interactions with the extension itself (e.g., extension usage frequency), rather than with the websites, are misclassified as the *User Activity* category. False negatives are primarily caused by data collection issues: one policy hosted on Google Docs is not properly crawled and thus no statements are extracted. In addition, we observe a single case where statements explicitly indicating local data processing are misinterpreted as data collection actions, thereby introducing both a false positive and a false negative.

We further evaluated the stability of the LLM-based analysis by repeating the analysis four more times and testing the different prompts on all 100 sampled policies. Across the total five runs, the mean precision was $95.67\% \pm 1.56\%$, while the mean recall was $97.42\% \pm 1.95\%$. Removing the scope rules decreased the precision to 87.07%, while the recall remained similar (97.58%) because the model extracted more statements outside the extension-specific scope. In comparison, removing the examples had a smaller effect, resulting in a precision of 92.74% and a recall of 96.44%.

Since ExtPrivA [32] does not release its dataset or trained model, we cannot perform a direct comparison. Therefore, we reproduce ExtPrivA based on the description in the paper for comparison. Overall, ExtPrivA identifies only 63 privacy statements across 26 extensions. Manual verification shows that 16 of these statements are false positives, yielding a precision of 74.6% and a recall of 20.89% (47/225). The false positives are mainly caused by the reliance on keyword-based filtering for identifying extension-related statements, as well as the inability to effectively handle non-English policies. Note that we observed a lower precision compared with the result reported in the paper (91.7%) and they did not evaluate recall. In the absence of publicly available code and datasets, the extensions used in our reproduction may not exactly align with those in the original study, making the observed discrepancy in results plausible. Nonetheless, the results still highlight the inherent limitations of the NLP-based approach.

4.5 Case Study

To further illustrate the privacy policy compliance issues in browser extensions, we present two case studies as follows.

Extension “iibpgpkhpfggipbacfejkloldhmiel”, a color picker extension with 60K users, is found to collect all links embedded in visited web pages. As some of the collected links contain unique user identifiers (e.g., from facebook.com), the resulting tainted data flows are classified as carrying *PII* and *Web Content*. The links are transmitted to admitad.com, which is a third-party advertising platform, for further processing. Such data usage is not disclosed in either the dashboard disclosures or the free-form privacy policies, and does not match the intended design of the extension.

Extension “mgmandbdaecndellojppohiekbhcie”, with 300 users, designed for capturing web pages through shortcut commands and building knowledge trees, declares the collection of *PII* and *Web Content*. In our experiments, however, the extension automatically collected the entire web pages—even without any user interaction—thereby transmitting sensitive information like geolocation and stored passwords to its backend APIs for further processing. This illustrates a broader issue. Extensions that capture full pages inherently collect multiple types of sensitive data, yet many only disclose generic collection of *Web Content* without specifically excluding sensitive information.

5 Discussion and Limitations

Underlying Extension Privacy Risks Some extensions request broad permissions (e.g., “<all_urls>”) but restrict target domains through runtime logic, which falls outside our current testing scope yet may still pose host-specific risks. Our LLM-based analysis operates on rendered page snapshots and may miss sensitive content introduced by dynamic DOM updates. In addition, we use a fixed execution window and may miss delayed or evasive data collection behaviors, a common trade-off in scalable dynamic analysis. Therefore, our findings should be interpreted as a lower bound on privacy risks. Analyzing such behaviors would require longer-term monitoring across a broader range of runtime conditions, which we leave for future work.

Evaluation Scope and Generalizability Our evaluation is conducted on a large-scale set of unauthenticated web pages and a limited number of popular websites with test accounts, due to budget and resource constraints. As the analysis is fully automated, it is feasible to apply our approach more broadly, especially by browser vendors or regulatory bodies. The evaluation could also be extended to more authenticated websites by applying (semi-)automatic login techniques [43, 55], which we leave as future work. In addition, while our study centers on Chrome extensions, the design is not Chrome-specific and can be adapted to other browser extension architectures, such as Firefox and Edge.

6 Conclusion

In this work, we introduce EXT*POLICE*, an automated and scalable framework for detecting privacy policy violations in browser extensions. Unlike prior efforts that either focus narrowly on specific data types or rely on manual heuristics, EXT*POLICE* combines multi-type taint analysis with LLM-based web content annotation to achieve fine-grained and scalable detection of sensitive data flows. Applying EXT*POLICE* to ~150K Chrome extensions, we identify 2,246 extensions whose runtime behaviors violate their stated privacy practices, affecting up to tens of millions of users. These results expose systematic weaknesses in current extension vetting and disclosure mechanisms. We responsibly disclosed our findings to the Chrome Extension team.

Acknowledgments

The authors would like to thank Danyang He for contributing to this work. The work described in this paper was partly supported by two grants from the Research Grants Council of the Hong Kong SAR, China (Project No.: CUHK 14209124 and CUHK 14211325).

Data Availability Statement

The research artifacts supporting this work are publicly available at <https://doi.org/10.5281/zenodo.21761007>.

References

- [1] 2019. DataSpill - The catastrophic data leak via browser extensions — dataspii.com. <https://dataspii.com/>. [Accessed 25-05-2025].
- [2] 2020. Facebook sues two Chrome extension makers for scraping user data — zdnet.com. <https://www.zdnet.com/article/facebook-sues-two-chrome-extension-makers-for-scraping-user-data/>. [Accessed 06-06-2025].
- [3] 2021. Facebook sues makers of malicious Chrome extensions for scraping data. <https://www.bleepingcomputer.com/news/security/facebook-sues-makers-of-malicious-chrome-extensions-for-scraping-data/>. [Accessed 16-07-2026].
- [4] 2023. Manifest V3 migration guide — extensionworkshop.com. <https://extensionworkshop.com/documentation/develop/manifest-v3-migration-guide/>. [Accessed 19-03-2026].
- [5] 2023. "FakeGPT" #2: Open-Source Turned Malicious in Another Variant of the Facebook Account-Stealer Chrome Extension — guard.io. <https://guard.io/labs/fakegpt-2-open-source-turned-malicious-in-another-variant-of-the-facebook-account-stealer>. [Accessed 16-07-2026].
- [6] 2024. Browser market share: The most popular browsers of 2024 — sox.com. <https://sox.com/research/browser-market-share>. [Accessed 27-08-2025].
- [7] 2024. Migrate to Manifest V3 | Chrome for Developers — developer.chrome.com. <https://developer.chrome.com/docs/extensions/develop/migrate>. [Accessed 19-03-2026].
- [8] 2025. Add-on Policies — extensionworkshop.com. <https://extensionworkshop.com/documentation/publish/add-on-policies/>. [Accessed 06-06-2025].
- [9] 2025. Add-ons for Firefox (en-US) — addons.mozilla.org. <https://addons.mozilla.org/en-US/firefox/>. [Accessed 27-08-2025].
- [10] 2025. Blink (Rendering Engine) — chromium.org. <https://www.chromium.org/blink/>. [Accessed 27-08-2025].
- [11] 2025. Catapult - Web Page Replay — chromium.googlesource.com. https://chromium.googlesource.com/catapult/+HEAD/web_page_replay_go/README.md. [Accessed 29-04-2025].
- [12] 2025. Chrome Web Store — chromewebstore.google.com. <https://chromewebstore.google.com/>. [Accessed 29-04-2025].
- [13] 2025. DarkSpectre: Unmasking the Threat Actor Behind 8.8 Million Infected Browsers — koi.ai. <https://www.koi.ai/blog/darkspectre-unmasking-the-threat-actor-behind-7-8-million-infected-browsers>. [Accessed 16-07-2026].
- [14] 2025. Google and Microsoft Trusted Them. 2.3 Million Users Installed Them. They Were Malware. — koi.ai. <https://www.koi.ai/blog/google-and-microsoft-trusted-them-2-3-million-users-installed-them-they-were-malware>. [Accessed 16-07-2026].
- [15] 2025. KeyboardEvent - Web APIs | MDN — developer.mozilla.org. <https://developer.mozilla.org/en-US/docs/Web/API/KeyboardEvent>. [Accessed 25-05-2025].
- [16] 2025. SpyVPN: The Google-Featured VPN That Secretly Captures Your Screen — koi.ai. <https://www.koi.ai/blog/spyvpn-the-vpn-that-secretly-captures-your-screen>. [Accessed 16-07-2026].
- [17] 2025. Tech Note - Malicious browser extensions impacting at least 3.2 million users - GitLab Security Tech Notes — gitlab.com.gitlab.io. <https://gitlab.com/gitlab-io/gl-security/security-tech-notes/threat-intelligence-tech-notes/malicious-browser-extensions-feb-2025/>. [Accessed 16-07-2026].
- [18] 2025. Updated Privacy Policy & Secure Handling Requirements | Chrome Web Store - Program Policies | Chrome for Developers — developer.chrome.com. <https://developer.chrome.com/docs/webstore/program-policies/user-data-faq>. [Accessed 30-05-2025].
- [19] 2025. V8 JavaScript engine — v8.dev. <https://v8.dev/>. [Accessed 27-08-2025].
- [20] 2025. XPath - XML: Extensible Markup Language | MDN — developer.mozilla.org. <https://developer.mozilla.org/en-US/docs/Web/XML/XPath>. [Accessed 01-06-2025].
- [21] 2026. AiFrame- Fake AI Assistant Extensions Targeting 260,000 Chrome Users via injected iframes. <https://layerxsecurity.com/blog/aiframe-fake-ai-assistant-extensions-targeting-260000-chrome-users-via-injected-iframes/>. [Accessed 16-07-2026].
- [22] 2026. From a Sophisticated Browser-Extension Supply-Chain Compromise to a VibeCoded Twist: A Chrome Extension as the Initial Access Vector for a Broader Malware Chain — monxresearch-sec.github.io. <https://monxresearch-sec.github.io/shotbird-extension-malware-report/>. [Accessed 16-07-2026].
- [23] 2026. How We Discovered A Campaign of 16 Malicious Extensions Built to Steal ChatGPT Accounts. <https://layerxsecurity.com/blog/how-we-discovered-a-campaign-of-16-malicious-extensions-chatgpt/>. [Accessed 16-07-2026].
- [24] Shubham Agarwal, Aurore Fass, and Ben Stock. 2024. Peeking through the window: Fingerprinting browser extensions through page-visible execution traces and interactions. In *Proceedings of the 2024 ACM SIGSAC conference on computer and communications security*. 2117–2131. doi:10.1145/3658644.3670339
- [25] Anupama Aggarwal, Bimal Viswanath, Liang Zhang, Saravana Kumar, Ayush Shah, and Ponnurangam Kumaraguru. 2018. I spy with my little eye: Analysis and detection of spying browser extensions. In *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 47–61. doi:10.1109/eurosp.2018.00012
- [26] Benjamin Andow, Samin Yaseer Mahmud, Wenyu Wang, Justin Whitaker, William Enck, Bradley Reaves, Kapil Singh, and Tao Xie. 2019. {PolicyLint}: investigating internal privacy policy contradictions on google play. In *28th USENIX security symposium (USENIX security 19)*. 585–602.
- [27] Benjamin Andow, Samin Yaseer Mahmud, Justin Whitaker, William Enck, Bradley Reaves, Kapil Singh, and Serge Egelman. 2020. Actions speak louder than words: {Entity-Sensitive} privacy policy and data flow analysis with {PoliCheck}. In *29th USENIX Security Symposium (USENIX Security 20)*. 985–1002.
- [28] Kirill Boychenko. 2026. Malicious Chrome Extension Steals MEXC API Keys for Account ... — socket.dev. <https://socket.dev/blog/malicious-chrome-extension-steals-mexc-api-keys>. [Accessed 16-07-2026].
- [29] Martin Brinkmann. 2019. Chrome Web Store has 188k extensions with at least 1.2 billion installs - gHacks Tech News — ghacks.net. <https://www.ghacks.net/2019/08/04/chrome-web-store-has-188k-extensions-with-at-least-1-2-billion-installs/>. [Accessed 29-04-2025].
- [30] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [31] Duc Bui, Kang G Shin, Jong-Min Choi, and Junbum Shin. 2021. Automated extraction and presentation of data practices in privacy policies. *Proceedings on Privacy Enhancing Technologies* (2021). doi:10.2478/popets-2021-0019
- [32] Duc Bui, Brian Tang, and Kang G Shin. 2023. Detection of inconsistencies in privacy practices of browser extensions. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2780–2798. doi:10.1109/sp46215.2023.10179338
- [33] Duc Bui, Yuan Yao, Kang G Shin, Jong-Min Choi, and Junbum Shin. 2021. Consistency analysis of data-usage purposes in mobile apps. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. 2824–2843. doi:10.1145/3460120.3484536
- [34] Moshe Siman Tov Bustan. 2025. 900K Users Compromised: Chrome Extensions Steal ChatGPT and DeepSeek Conversations - OX Security — ox.security. <https://www.ox.security/blog/malicious-chrome-extensions-steal-chatgpt-deepseek-conversations/>. [Accessed 16-07-2026].
- [35] Ahmet Salih Buyukkayhan, Kaan Onarlioglu, William K Robertson, and Engin Kirda. 2016. CrossFire: An Analysis of Firefox Extension-Reuse Vulnerabilities.. In *NDSS*. doi:10.14722/ndss.2016.23149
- [36] Stefano Calzavara, Michele Bugliesi, Silvia Crafa, and Enrico Steffanlongo. 2015. Fine-grained detection of privilege escalation attacks on browser extensions. In *Programming Languages and Systems: 24th European Symposium on Programming, ESOP 2015, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2015, London, UK, April 11-18, 2015, Proceedings* 24. Springer, 510–534. doi:10.1007/978-3-662-46669-8_21
- [37] Wentao Chang and Songqing Chen. 2016. ExtensionGuard: Towards runtime browser extension information leakage detection. In *2016 IEEE Conference on Communications and Network Security (CNS)*. IEEE, 154–162. doi:10.1109/cns.2016.7860481
- [38] Quan Chen and Alexandros Kapravelos. 2018. Mystique: Uncovering information leakage from browser extensions. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. 1687–1700. doi:10.1145/3243734.3243823
- [39] Aurore Fass, Dolière Francis Somé, Michael Backes, and Ben Stock. 2021. Doublex: Statically detecting vulnerable data flows in browser extensions at scale. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. 1789–1804. doi:10.1145/3460120.3484745
- [40] Leon Garza, Lavanya Elluri, Aritrnan Pipalai, Anantaa Kotal, Deepti Gupta, and Anupam Joshi. 2024. PrivComp-KG: Leveraging KG and LLM for compliance verification. In *2024 IEEE 6th International Conference on Trust, Privacy and Security in Intelligent Systems, and Applications (TPS-ISA)*. IEEE, 97–106. doi:10.1109/tps-isa62245.2024.00021
- [41] Guardio. 2023. "FakeGPT": New Variant of Fake-ChatGPT Chrome Extension Stealing Facebook Ad Accounts with... — labs.guard.io. <https://labs.guard.io/fakegpt-new-variant-of-fake-chatgpt-chrome-extension-stealing-facebook-ad-accounts-with-4c9996a8f282>. [Accessed 06-06-2025].
- [42] Sheryl Hsu, Manda Tran, and Aurore Fass. 2024. What is in the chrome web store?. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*. 785–798. doi:10.1145/3634737.3637636
- [43] Hugo Jonker, Stefan Karsch, Benjamin Krumnow, and Marc Slegers. 2020. Shepherd: a generic approach to automating website login. (2020). doi:10.14722/madweb.2020.23008
- [44] Alexandros Kapravelos, Chris Grier, Neha Chachra, Christopher Kruegel, Giovanni Vigna, and Vern Paxson. 2014. Hulk: Eliciting malicious behavior in browser extensions. In *23rd USENIX Security Symposium (USENIX Security 14)*. 641–654.
- [45] Pierre Laperdix, Oleksii Starov, Quan Chen, Alexandros Kapravelos, and Nick Nikiforakis. 2021. Fingerprinting in style: Detecting browser extensions via

- injected style sheets. In *30th USENIX security symposium (USENIX Security 21)*. 2507–2524.
- [46] Yuxi Ling, Yun Hao, Yuyan Wang, Kailong Wang, Guangdong Bai, and Jin Song Dong. 2024. Essential or Excessive? MINDAEXT: Measuring Data Minimization Practices among Browser Extensions. In *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 964–975. doi:10.1109/saner60148.2024.00104
- [47] Yuxi Ling, Kailong Wang, Guangdong Bai, Haoyu Wang, and Jin Song Dong. 2022. Are they toeing the line? diagnosing privacy compliance violations among browser extensions. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–12. doi:10.1145/3551349.3560436
- [48] mingabire. 2021. Combating Scraping by Malicious Browser Extensions – about.fb.com. <https://about.fb.com/news/2021/01/combating-scraping-by-malicious-browser-extensions/>. [Accessed 19-05-2025].
- [49] Asmit Nayak, Rishabh Khandelwal, Earlene Fernandes, and Kassem Fawaz. 2024. Experimental security analysis of sensitive data access by browser extensions. In *Proceedings of the ACM Web Conference 2024*. 1283–1294. doi:10.1145/3589334.3645683
- [50] Kush Pandya. 2026. 5 Malicious Chrome Extensions Enable Session Hijacking in En... – socket.dev. <https://socket.dev/blog/5-malicious-chrome-extensions-enable-session-hijacking>. [Accessed 16-07-2026].
- [51] Kush Pandya. 2026. Chrome and Firefox Extensions Posing as Free VPNs Add Clipbo... – socket.dev. <https://socket.dev/blog/chrome-and-firefox-extensions-free-vpns-add-clipboard-stealers>. [Accessed 16-07-2026].
- [52] Nikolaos Pantelaios, Nick Nikiforakis, and Alexandros Kapravelos. 2020. You’ve changed: Detecting malicious browser extensions through their update deltas. In *Proceedings of the 2020 ACM SIGSAC conference on computer and communications security*. 477–491. doi:10.1145/3372297.3423343
- [53] Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*. 1–22. doi:10.1145/3586183.3606763
- [54] Tamjid Al Rahat, Minjun Long, and Yuan Tian. 2022. Is your policy compliant? a deep learning-based empirical study of privacy policies’ compliance with gdpr. In *Proceedings of the 21st Workshop on Privacy in the Electronic Society*. 89–102. doi:10.1145/3559613.3563195
- [55] Jannis Rautenstrauch, Metodi Mitkov, Thomas Helbrecht, Lorenz Hetterich, and Ben Stock. 2024. To auth or not to auth? a comparative analysis of the pre-and post-login security landscape. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1500–1516. doi:10.1109/sp54263.2024.00094
- [56] Dolière Francis Somé. 2019. EmPoWeb: empowering web applications with browser extensions. In *2019 IEEE Symposium on Security and Privacy (SP)*. IEEE, 227–245. doi:10.1109/sp.2019.00058
- [57] Oleksii Starov and Nick Nikiforakis. 2017. Extended tracking powers: Measuring the privacy diffusion enabled by browser extensions. In *Proceedings of the 26th International Conference on World Wide Web*. 1481–1490. doi:10.1145/3038912.3052596
- [58] Oleksii Starov and Nick Nikiforakis. 2017. Xhound: Quantifying the fingerprintability of browser extensions. In *2017 IEEE Symposium on Security and Privacy (SP)*. IEEE, 941–956. doi:10.1109/sp.2017.18
- [59] 7AI Threat Research Team. 2026. CRXfiltrate: An Undocumented JavaScript Execution Backdoor in a Chrome Extension Network – 7ai.com. <https://7ai.com/crxfiltrate>. [Accessed 16-07-2026].
- [60] Michael Weissbacher, Enrico Mariconti, Guillermo Suarez-Tangil, Gianluca Stringhini, William Robertson, and Engin Kirda. 2017. Ex-ray: Detection of history-leaking browser extensions. In *Proceedings of the 33rd Annual Computer Security Applications Conference*. 590–602. doi:10.1145/3134600.3134632
- [61] Mengfei Xie, Jianming Fu, Jia He, Chenke Luo, and Guojun Peng. 2020. JTaint: finding privacy-leakage in chrome extensions. In *Information Security and Privacy: 25th Australasian Conference, ACISP 2020, Perth, WA, Australia, November 30–December 2, 2020, Proceedings 25*. Springer, 563–583. doi:10.1007/978-3-030-55304-3_29
- [62] Qinge Xie, Manoj Vignesh Kasi Murali, Paul Pearce, and Frank Li. 2024. Arcanum: detecting and evaluating the privacy risks of browser extensions on web pages and web content. In *33rd USENIX Security Symposium (USENIX Security 24)*. 4607–4624.
- [63] Qinge Xie, Karthik Ramakrishnan, and Frank Li. 2025. Evaluating privacy policies under modern privacy laws at scale: An {LLM-Based} automated approach. In *34th USENIX Security Symposium (USENIX Security 25)*. 5797–5816.
- [64] Jianjia Yu, Song Li, Junmin Zhu, and Yinzhi Cao. 2023. CoCo: Efficient Browser Extension Vulnerability Detection via Coverage-guided, Concurrent Abstract Interpretation. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 2441–2455. doi:10.1145/3576915.3616584
- [65] Yufei Zhao, Longtao He, Zhoujun Li, Liquan Yang, Hao Dong, Chao Li, and Yu Wang. 2019. Large-scale detection of privacy leaks for BAT browsers extensions in China. In *2019 International Symposium on Theoretical Aspects of Software Engineering (TASE)*. IEEE, 57–64. doi:10.1109/tase.2019.00-19
- [66] Lu Zhou, Chengyongxiao Wei, Tong Zhu, Guoxing Chen, Xiaokuan Zhang, Suguo Du, Hui Cao, and Haojin Zhu. 2023. {POLICYCOMP}: counterpart comparison of privacy policies uncovers overbroad personal data collection practices. In *32nd USENIX Security Symposium (USENIX Security 23)*. 1073–1090.
- [67] Sebastian Zimmeck, Ziqi Wang, Lieyong Zou, Roger Iyengar, Bin Liu, Florian Schaub, Shomir Wilson, Norman M Sadeh, Steven M Bellovin, and Joel R Reidenberg. 2017. Automated Analysis of Privacy Requirements for Mobile Apps.. In *NDSS*, Vol. 2. 1–4. doi:10.14722/ndss.2017.23034

Received 2026-03-27; accepted 2026-06-18